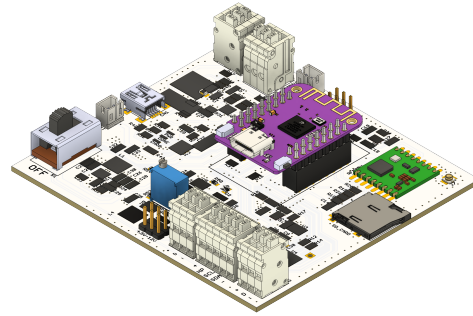


WiLo V2

(Wireless logger)



Source files available here → [WiLo Git](#)

Overview

- I2C, SDI-12, One-Wire communication protocols
- Minimum 40 day battery life when connected to 3.7 V 5000 mAh Li-Po battery
- Environmental Sensing
- LoRa network integration
- Local SD storage
- Solar powered
- Solar panel over-voltage protection
- Highly accurate RTC (DS3231) for timestamps
- Fixed 3.3 V onboard power supply
- Adjustable 4.7–12 V power supply
- Push button terminals for sensors

Description

- The device is intended for field deployment, to measure various environmental metrics and/or use in precision forestry applications where tree growth and water dynamics must be measured.
- An SDI-12 terminal allows integration with SDI-12 sensors. ¹
- One-Wire sensors (such as DS18B20 temperature sensor or DHT22 Temperature and humidity sensor) have dedicated terminals.
- I2C and UART capable.
- Battery voltage detection allows user to measure current battery voltage level.

¹At the time of writing, the firmware of WiLoV2 only supports Sentek 60 cm and 90 cm Sentek Drill and Drop Series III probes, and the CS655 soil moisture probe.

Contents

1	Technical specification	4
2	Board Overview	4
2.1	Sensor port wiring guide	4
3	System Processes	5
3.1	Initial Startup	5
3.1.1	Activate WiFi capabilities for OTA (Over-the-Air) update and setting RTC . .	5
3.1.2	Set CPU frequency to 40 MHz	5
3.1.3	Initialise SD Card	5
3.1.4	Read/Write files from/to SD card	5
3.1.5	I2C Device Check	6
3.2	Measurements	7
3.2.1	Battery	7
3.2.2	Air Temperature and Humidity (DHT22)	7
3.2.3	Soil Temperature (DS18B20)	7
3.2.4	Soil Moisture (Capacitive Soil Moisture V1.2) / Apogee Pyronometer	7
3.2.5	Measure Dendrometer (DigiDendro)	8
3.2.6	Measure SU Forestry Sapflow (SF)	8
3.2.7	SDI-12	8
3.3	Write Data to SD Card	9
3.4	LoRa Setup	9
3.5	Wake-up Procedures	9
4	Changing WiLo V2 Transmission Intervals	9
4.1	Payload Size Overview	10
4.2	Payload Size Tables for Drill & Drop Configurations	10
4.3	Best Practice Recommendations	11
4.4	Recommended Transmission Intervals	11
5	SDI-12 Installation and Configuration	12
5.1	SDI-12 Memory Addressing	12
5.2	SDI-12 Voltage Configuration	12
6	SD Card	14
6.1	Formatting SD card	14
6.2	SD Card Measurement Files	14
7	Firmware Upgrade	14
7.1	Configuring PlatformIO and Source Files in VS Code	15
7.2	Physical Connection Firmware Upgrade	15
7.3	Wireless Connection (OTA) Firmware Upgrade	16
7.3.1	Retrieving firmware.bin File	16
7.3.2	Setting up mobile device/PC for OTA	16
8	Debugging	18
8.1	Hardware Debugging	18
8.1.1	Battery	18

8.1.2	SD Card Errors	19
8.2	Sensor Errors	19
9	Adding WiLos to The Things Network	19
9.1	Adding a single WiLo	19
9.2	Bulk Adding Wilos	22

1 Technical specification

	Unit	Value
Battery Voltage	V	3.7 V
Rail Voltage	V	3.3 V
SDI-12 Boost Voltage	V	4.5–12 V
Maximum Active Current Draw	mA	400
Maximum Sleep Current Draw	μ A	500
Dimensions	mm $\times$ mm $\times$ mm	72x95x40

2 Board Overview

Careful consideration is required when connecting sensors and inputs to the WiLo ports (Fig. 1).

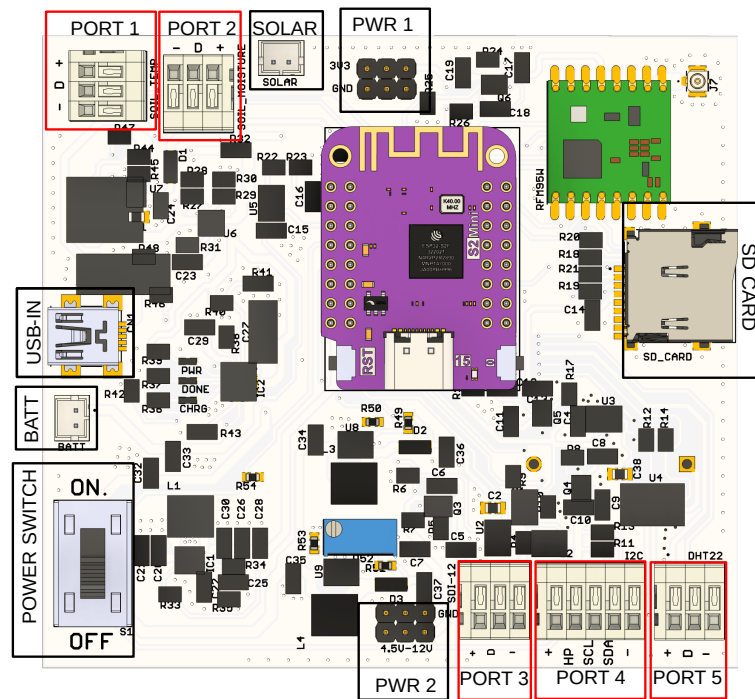


Figure 1: WiLo V2 board map. Sensors are connected to push terminals, labelled PORT 1 – PORT 4 (outlined in red). PORT 1: 18B20S temperature for soil temperature measurements. PORT 2: Capacitive Soil Moisture Probe V1.2, or Apogee Pyranometer. PORT 3: SDI-12 sensors. PORT 4: I2C Port for I2C devices (Gravity: Digital 1-to-8 I2C Multiplexer, SU Forestry Sapflow, DigiDendro. PORT5: DHT22 relative humidity and temperature sensor. SOLAR: Solar panel JST-PH connector. PWR 1: Male headers for testing the 3.3 V rail, or powering external peripherals. USB-IN: A micro-usb port to power the system and recharge the battery with a 5 V 500 mA source. BATT: JST-PH connector for a 3.7 V Li-Po battery. POWER SWITCH: The main power switch, allowing power from the battery to the system. SD CARD: The and SD card slot for local SD storage backup. PWR2: Male headers of the 4.5–12 V adjustable power supply.

2.1 Sensor port wiring guide

Table 12 describes the pinmapping to MCU pins of the ESP32-S2 mini microcontroller. These pins are defined within the Firmware of the WiLo. The pinmapping can be found in the `config.h` header file of the WiLo source code, in the `Wilov2 Code/include` folder.

Table 1: Terminal layout for the WiLo V2 board, showing the sensor connections for PORT 1–PORT 5 as labelled on the board. Each port includes push terminals marked + (Power), D (Data), and – (Ground), with PORT 4 also including SCL, SDA, and HP for I²C and sapflow devices. Terminal functions and corresponding wire colours are listed for typical sensors.

Port	Terminal Label	Signal	Sensor Wire Colour
PORT 1	+	Power 3.3 V	RD
	D	DATA (18B20S, soil temperature)	YL/WH
	–	GND	BK
PORT 2	+	Power 3.3 V	RD
	D	Capacitive SM / Pyranometer DATA	YL/WH
	–	GND	BK
PORT 3	+	Power 4.5–12 V	RD
	D	SDI-12 DATA	WH
	–	GND	BK
PORT 4 (I2C)	+	Power 3.3 V	RD
	HP	Sapflow Heat Pulse Trigger	YL/WH
	SCL	I2C device SCL (e.g. Multiplexer, DigiDendro, SF)	YL
	SDA	I2C Device SDA (e.g. Multiplexer, DigiDendro, SF)	BL
	–	GND	BK
PORT 5	+	Power 3.3 V	RD
	D	DHT22 DATA	YL/WH
	–	GND	BK

3 System Processes

The following section describes the systems processes during a single cycle.

3.1 Initial Startup

3.1.1 Activate WiFi capabilities for OTA (Over-the-Air) update and setting RTC

OTA update allows firmware to be loaded onto the ESP32 board wirelessly, using a Wi-Fi connection. For more information on updating device firmware, see Section 7. The WiFi window stays open for 15 seconds in attempt to allow a WiFi capable device to connect, after which WiFi capabilities of the WiLo is disabled, until the device is reset or loses power.

3.1.2 Set CPU frequency to 40 MHz

This slows down the CPU to reduce active-mode current draw.

3.1.3 Initialise SD Card

The device attempts to initialise the SD card module. If SD card initialisation is unsuccessful, the debug LED on the Lolin ESP32-S2 mini will flash for five seconds, then go into deep sleep for ten seconds.

3.1.4 Read/Write files from/to SD card

The system checks if a file called `Measurements.csv` is on the SD card. If not, the file is created. See Section 6 for interpretation of the files on the SD card. At this point the LoRa parameters are read from the file called `parameters.txt`. If no such file exists, or the system fails to read the content, the system goes into an infinite loop, where the debug LED stays ON for 3 seconds, and OFF for three seconds.

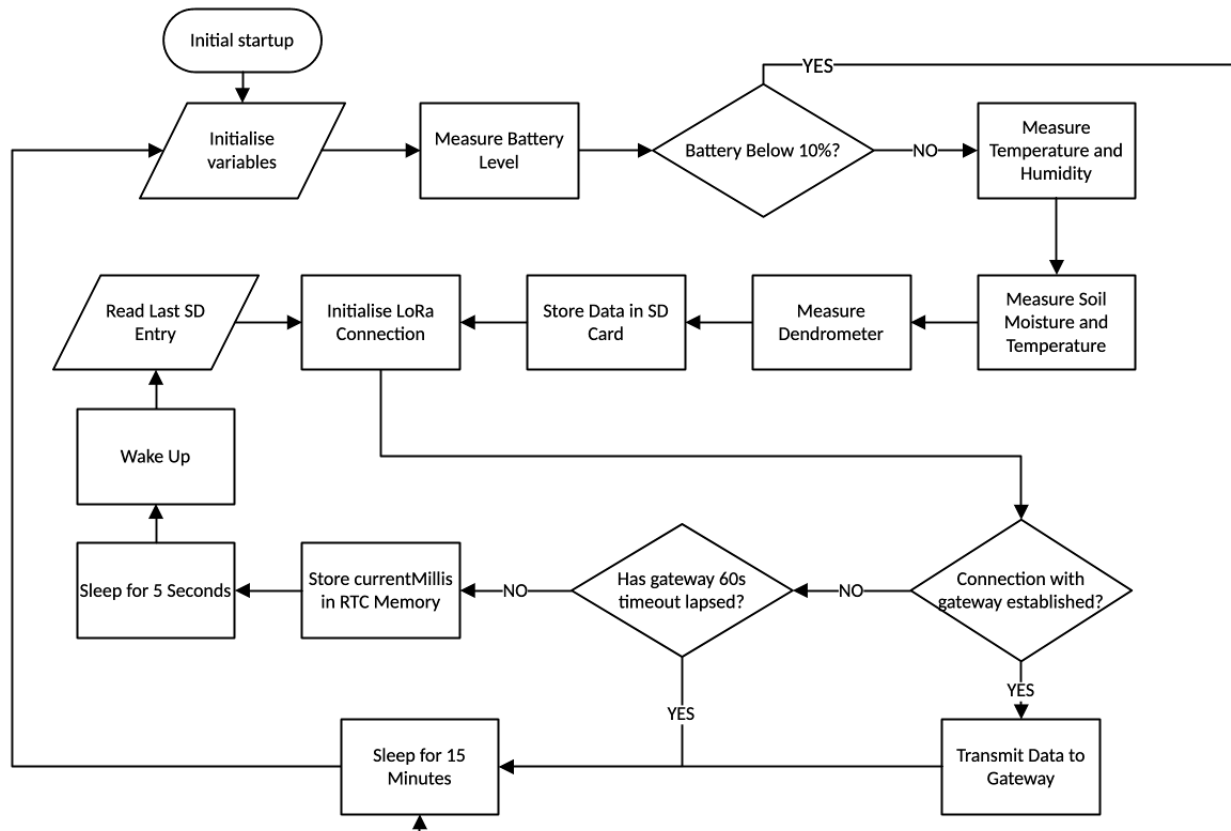


Figure 2: WiLo system high-level flow diagram

3.1.5 I2C Device Check

The system scans the I2C port (PORT 4) to determine which I2C device is connected.

Table 2: Different I2C devices supported by the current firmware of WiLo V2

Sensor	Description	Device Address
DigiDendro (DD)	Digitised dendrometer	0x68
SU Forestry Sapflow (SF)	Heatpulse velocity method sapflow sensor	0x40, 0x41
Gravity: I2C MUX	An I2C MUX that expands the I2C port (1:8), allowing up to eight I2C devices (DD or SF) to be connected to a single WiLo	0x70

To avoid I2C device address overlap, certain conditions must be met:

- If an I2C MUX is used, **no SF or DD may be connected to PORT 4 alongside the MUX.** Populate the ports on the I2C MUX.
- If no MUX is used, **only a single DD and a single SF may be connected on PORT 4.**

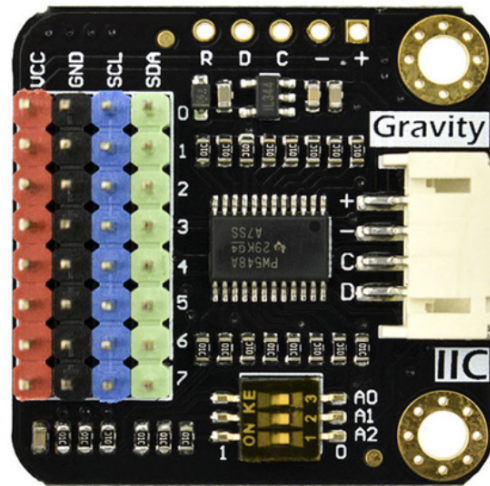


Figure 3: DFRobot Gravity I2C Mux. See ports labelled 0-7. Singular, multiple, or all ports can be populated with DigiDendro or Sapflow sensors (may be mixed). It is the user's responsibility to make note of which sensor is connected to which port. The payload formatter provided in Appendix A.

3.2 Measurements

All measurements are taken simultaneously instead of sequentially, in order to reduce system active time. For each sensor, a predefined number of samples (S) is taken. After sampling is done, the samples are ordered from smallest to largest, and the trimmed mean of a predefined number (N) of centremost values of the array are averaged, dropping significant outliers from the calculation. Note that in the case of a faulty sensor, different value codes are given to sensor measurements (see Section 9).

3.2.1 Battery

The battery voltage is detected. 3.7 V Li-Po batteries have an output voltage ranging from 4.2 V at full charge (100 %) to around 3 V at depletion (0 %). $S = 20$, $N = 5$.

3.2.2 Air Temperature and Humidity (DHT22)

The DHT22 measurement is started. Temperature is returned in $^{\circ}\text{C}$ and relative humidity is returned as a percentage (%). $S = 10$, $N = 5$.

3.2.3 Soil Temperature (DS18B20)

Similarly, the measurements of the DS18B20 sensor are taken, returned as $^{\circ}\text{C}$. $S = 5$, $N = 3$.

3.2.4 Soil Moisture (Capacitive Soil Moisture V1.2) / Apogee Pyronometer

The Soil moisture probe as well as the Pyranometer are analogue devices. As there is no way to separate the two signals, **only one of either sensors may be connected at a time**. The output gives a raw voltage measurement, that must be converted to readable data. $S = 10$, $N = 5$.

3.2.5 Measure Dendrometer (DigiDendro)

Dendrometer measurements are taken. Dendrometer measurements are given as an absolute value, ranging between $0 \mu\text{m}$ and $9999.99 \mu\text{m}$. If no dendro is connected, the system simply returns 0 as the dendrometer measurement. $S = 10$, $N = 5$.

3.2.6 Measure SU Forestry Sapflow (SF)

The Sapflow sensor has the longest measurement time of all connected sensors. The upstream temperature (T_1) and downstream temperature (T_2) are sampled every second for 10 seconds, while the heatpulse is off. Then, the heating element is turned on, heating the stem, and T_1 and T_2 are sampled every second for 20 seconds. Finally, the heating element is turned off and T_1 and T_2 are sampled for an additional 10 seconds. The values of T_1 and T_2 are averaged for each state. The Heat Pulse Velocity (HPV) can be calculated based on the Marshall formula for HPV:

$$\text{HPV} = \frac{k}{x} \cdot \ln \left(\frac{\Delta T_2}{\Delta T_1} \right) \cdot 3600 \quad (1)$$

Where:

- HPV : Heat Pulse Velocity (mm/h), the velocity at which a thermal pulse moves through the medium.
- k : Thermal diffusivity constant (mm^2), representing the effective cross-sectional area involved in heat conduction.
- x : Radial distance between the heater and the temperature sensors (mm).
- $\Delta T_1 = T_{1,\text{after}} - T_{1,\text{before}}$: Average temperature increase before and after the heat pulse at sensor 1 (upstream).
- $\Delta T_2 = T_{2,\text{after}} - T_{2,\text{before}}$: Average temperature increase before and after the heat pulse at sensor 2 (downstream).
- 3600 : Conversion factor from seconds to hours.

If a MUX is used, the same timings for the heatpulse state (before, during and after are used). The heatpulse states of all the SF sensors connected to the mux are the same at any given moment.

3.2.7 SDI-12

The WiLo checks to see if an SDI-12 device is connected. Note that the firmware only supports the Sentek SDI-12 Series III 90 cm and 60 cm drill and drop probes, and the Campbell Scientific CS655 Probe. Each SDI-12 device needs its address to be configured *before* connecting to the WiLo. Refer to Section 5 for installation and configuration considerations. If a SDI-12 device is connected, a file will be created, depending on the SDI-12 device connected. File names are as follows:

- **Sentek SDI-12 Series III 90 cm:** `SDI_12_90cm_Measurements.csv`
- **Sentek SDI-12 Series III 60 cm:** `SDI_12_60cm_Measurements.csv`
- **Campbell Scientific CS655:** `SDI_12_CS655_Measurements.csv`

if there is no SDI-12 device detected, the WiLo will simply omit the measurements from the payload.

3.3 Write Data to SD Card

Standard WiLo measurements are written to `Measurements.csv`, SDI-12 Measurements (if the device is connected) is written to the corresponding SDI-12 `SDI12_X_Measurements.csv` file.

If an I2C MUX is connected, data is written to two files, namely

`I2C_Mux_Dendrometer_Measurements.csv` and `I2C_Mux_Sapflow_Measurements.csv`

3.4 LoRa Setup

Once all measurements are done, the WiLo creates a LoRa payload. The payload is passed on to the RFM95W LoRa module, which attempts to connect to a nearby LoRa gateway using the keys stored in the `parameters.txt` file. If a successful connection is made, the LoRa payload is transmitted to the gateway and the WiLo goes into deep sleep. In the case of no connection, the device goes into a light sleep for 5 seconds, wakes up, attempts to transmit the payload again. This is done until a timer reaches 60 seconds, after which the device goes to deep sleep.

3.5 Wake-up Procedures

After an active cycle of measuring, storing data in the SD card and transmitting data over LoRa, the WiLo wakes up after some time. In the current firmware, the WiLo keeps track of the active time and adjusts the deep sleep time accordingly. The total cycle time is 20 minutes, in order to keep measurements approximately 20 minutes apart. Total cycle time can be expressed as:

$$\text{Cycle Time} = \text{Active Time} + \text{Deep Sleep Time} = 20 \text{ minutes} \quad (2)$$

For each consecutive wake-up, the device runs through processes 3.1 – 3.4.

4 Changing WiLo V2 Transmission Intervals

Depending on the application, the user may want to change **WiLo Cycle Time**. To do so, navigate to `WiLoV2/WiLoV2 Code/include`. Inside this folder, there is a file called `config.h`. Open this file with a text editor. Under the `//Deep sleep times` section, you will see a variable defined as `#define WIL0_MAX_CYCLE_TIME 1200`. This is the maximum cycle time of the WiLo in seconds. If the WiLo is active for time x , deep sleep is thus `WIL0_MAX_CYCLE_TIME - x`. Change this value to suit the application.

Be aware that The Things Network enforces a fair use policy. This means that a WiLo is allowed up to 30s of over-the-air time per day. In essence, the total time it takes for a WiLo to send payloads to the gateway must not exceed 30s in one day. The payload size is also a limitation enforced by the LoRa free-use policy. Payloads may not exceed **222 bytes**.

TTN provides a useful [LoRaWAN Airtime Calculator](#) to calculate what the airtime will be for a given payload. The parameters that can be entered in the calculator are as follows:

- **Input Bytes:** How many bytes do you want to send – See Tables 3 and 4
- **Spreading Factor:** SF7
- **Region:** EU868
- **Bandwidth:** 125 kHz

4.1 Payload Size Overview

The WiLo always transmits at least 13 fixed sensor values, including air/soil temperature, humidity, battery level, and boot count. Each of these values is 3 bytes. Additionally, 3 bytes are used for internal configuration flags. Therefore, the absolute minimum payload size is:

$$13 \text{ values} \times 3 \text{ bytes} + 3 \text{ config bytes} = \boxed{42 \text{ bytes}}$$

This is the base payload size for a WiLo with:

- No SDI-12 sensors connected
- No MUX sensors connected

When additional sensors are connected (e.g., a multiplexer with dendrometers or sap flow sensors), the payload size increases, and can approach or exceed the LoRaWAN limit of 222 bytes. The next section provides lookup tables for determining payload sizes for different combinations.

4.2 Payload Size Tables for Drill & Drop Configurations

The tables below show the number of bytes transmitted for various combinations of dendrometers and sap flow sensors connected to a WiLo device via I²C multiplexer ports. The WiLo supports a maximum of 8 sensor ports. The two configurations represent:

- A 90 cm Series III Sentek Drill & Drop probe (Table 3)
- A 60 cm Series III Sentek Drill & Drop probe (Table 4)

Any payload size above 222 bytes is invalid and cannot be transmitted over LoRaWAN.

Table 3: Payload size (in bytes) for 90 cm Sentek Drill & Drop probe

		Dendrometer Count								
		0	1	2	3	4	5	6	7	8
SapFlow Count	0	96	99	102	105	108	111	114	117	120
	1	114	117	120	123	126	129	132	135	
	2	132	135	138	141	144	147	150		
	3	150	153	156	159	162	165			
	4	168	171	174	177	180				
	5	186	189	192	195					
	6	204	207	210						
	7	222	225							
	8	240								

Table 4: Payload size (in bytes) for 60 cm Sentek Drill & Drop probe

		Dendrometer Count								
		0	1	2	3	4	5	6	7	8
SapFlow	0	78	81	84	87	90	93	96	99	102
	1	96	99	102	105	108	111	114	117	
	2	114	117	120	123	126	129	132		
	3	132	135	138	141	144	147			
	4	150	153	156	159	162				
	5	168	171	174	177					
	6	186	189	192						
	7	204	207							
	8	222								

4.3 Best Practice Recommendations

- For **90 cm probes**, avoid combinations that exceed 222 bytes. Examples:
 - **Valid:** 7 sap flow sensors + 0 dendrometers (222 bytes)
 - **Invalid:** 8 sap flow sensors (240 bytes), or 7 sap + 1 dendro (225 bytes)
- For **60 cm probes**, all combinations up to 8 sensors are safe (≤ 222 bytes).

4.4 Recommended Transmission Intervals

The Things Network (TTN) enforces a fair access policy that limits each LoRaWAN device to approximately **30 seconds of airtime per day**. Exceeding this limit may result in dropped packets or gateway throttling. We recommend always using a 20-minute interval, however in some applications a higher transmission frequency is desirable.

To simplify compliance, Table 5 below recommends safe transmission intervals based on estimated airtime for SF7. These assume no confirmed uplinks and minimal MAC-layer overhead. Values are conservative.

Table 5: Recommended WiLo transmission intervals based on payload size (SF7, EU868)

Payload Size (Bytes)	Estimated Airtime (ms)	Max Transmissions / Day	Recommended Interval (WILO_MAX_CYCLE_TIME)
42 (minimum config)	107.87	278	Every 6 minutes (360 s)
42–90	107.87–174.3	172	Every 10 minutes (600 s)
90–150	174.3–266.5	133	Every 15 minutes (900 s)
150–222	266.5–368.9	81	Every 20 minutes (1200 s)

Example: If your payload is 177 bytes (e.g., 4 sap flow + 3 dendrometers and a 90 cm probe), the estimated airtime is 302.3 ms. To remain under TTN’s 30s/day limit:

```
#define WILO_MAX_CYCLE_TIME 900
```

This sets a 15-minute interval, allowing about 96 transmissions/day — well within the safe zone.

5 SDI-12 Installation and Configuration

Before installation, the SDI-12 device must be programmed with a dedicated device address. The user must also adjust the power supply voltage to the SDI-12 device manually.

5.1 SDI-12 Memory Addressing

Table 6 shows the SDI-12 devices supported by the current WiLo version and the device addresses that must be written to the SDI-12 devices before installation. This step is critical in ensuring flawless measurements from these devices. Changing the device address can be done using an Arduino, or a Campbell scientific data logger.

- **Arduino:** The [Arduino-SDI-12 library](#) provides ample information, source code and examples on interfacing SDI-12 sensors to Arduino MCUs. **Warning: Arduinos can provide 5 V to the Sentek Series III drill and drop probes without issue, but other SDI-12 devices (such as the CS655) require 6–12 V power. Therefore, an external power supply is strongly recommended when writing memory addresses of SDI-12 devices that do not accept 5 V power. The 4.5–12 V adjustable power supply on the WiLo can be used to power an SDI12 sensor for memory addressing.**
- **Campbell Scientific dataloggers:** To assign a new memory address to an SDI-12 sensor using a Campbell Scientific data logger, you can use the logger’s terminal emulator to enter *transparent mode*. In this mode, send the command “aAb!”, where a is the current sensor address and b is the desired new address. For example, to change the address from 0 to B, input “0AB!”. The sensor should respond with the new address, confirming the change. This method is supported by several Campbell Scientific loggers, such as the CR1000X, CR6, CR300, and others. For more information and detailed instructions, refer to the following resources:
 - [Changing an SDI-12 Address](#)
 - [SDI-12 Transparent Mode](#)
 - [Change Address Command \(aAb!\)](#)

SDI-12 Device	Required Address
Sentek SDI-12 Series III 90 cm	S
Sentek SDI-12 Series III 60 cm	J
Campbell Scientific CS655	6

Table 6: Current SDI-12 support and required memory addresses

5.2 SDI-12 Voltage Configuration

Before an SDI-12 sensor is connected to a WiLo, the voltage supply to the sensor must be configured manually.

You will require:

- A male to male dupont jumper cable.
- A digital multimeter for voltage measurements.
- A small flathead screwdriver for adjusting the potentiometer.

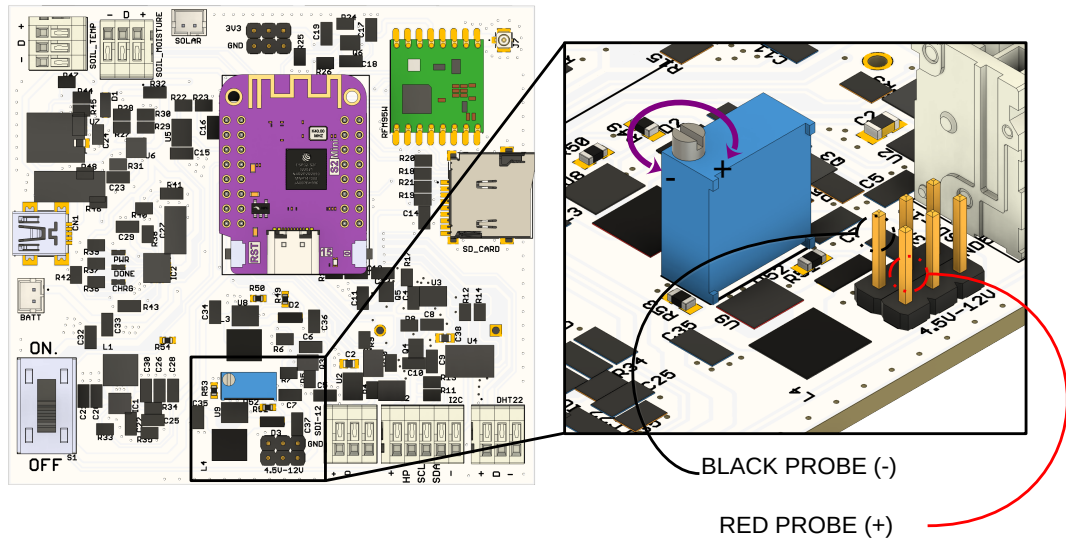


Figure 4: SDI-12 voltage configuration notes. The positive probe of a multimeter (usually red) must be connected to any of the three 4.5–12 V male headers. The negative probe (usually black) must be connected to any of the three the ground male headers. Turning the dial of the potentiometer clockwise increases the voltage (towards 12 V). Turning the dial counter-clockwise decreases the voltage towards 4.5 V.

The steps are as follows:

1. Turn off the WiLo by putting the power switch in the off position.
2. Remove the Lolin ESP32-S2 Mini from its headers on the WiLo.
3. Make sure the battery is properly connected. Turn on the WiLo by putting the power switch in the on position.
4. Set the multimeter to its voltage reading function, and place the positive probe (red) against the 4.5–12 V headers and the negative probe (black) against the ground headers (see Fig. 4). You should see a voltage level display on the multimeter.
5. Using the flathead screwdriver, turn the dial of the blue potentiometer clockwise to increase the boost voltage, and anticlockwise to decrease the boost voltage. **Do not overturn the potentiometer.** The minimum voltage required is specific to the SDI-12 probe used:
 - **Sentek Series III Probes:** 5 V.
 - **CS655:** 6 V.

Most SDI-12 devices accept 12 V power, so tuning the output to 12 V is also acceptable for both the Sentek Series III probes and the CS655.

6. Once the desired voltage level is reached, turn off the WiLo and resituate the Lolin ESP32-S2 mini in the headers.
7. Connect the SDI-12 device wires to the terminals according to the pin reference of Section 2.
8. Turn the WiLo on again and wait for approximately 3 minutes to see if the payload has been transmitted to TTN, or alternately perform a serial debug to see whether values were measured correctly.

6 SD Card

6.1 Formatting SD card

Before the WiLo can be deployed, the SD card must be formatted and the `LoRa parameters.txt` file must be copied to the SD card. Formatting is done with these steps:

1. Place the SD card in a micro-SD adapter and connect it to a PC.
2. format the SD card to FAT32. This is of utmost importance, as no other SD format is supported. Windows PCs do not allow SD card sizes greater than 32 GB to be formatted to FAT32. There is a workaround for this; using a third party tool. for a full guide, see <https://www.easeus.com/partition-master/format-64gb-sd-card-to-fat32.html>.
3. Copy the LoRa parameter file to the SD card. Backups can be found here: [GitHub Repository](#)
4. Insert the SD card in the SD card slot of the WiLo.

6.2 SD Card Measurement Files

On the SD card, there will be up to measurement files. The first, which is the default of all WiLos, is `Measurements.csv`.

Table 7: Descriptions of columns in the `Measurements.csv` file

Name	Description	Unit
Dendrometer	Digitised dendromter measurement	μm
Air Temperature	DHT22 air temperature at the Canopy	$^{\circ}\text{C}$
Air Humidity	DHT22 relative humidity at the canopy	%
SM or Pyro	Voltage output of the Capacitive Soil Moisture sensor or Apogee SP-110	mV
Battery	Battery pecentage	%
SF T1 Before	Average temperature of upstream sapflow temperature sensor (T1) before heat pulse	$^{\circ}\text{C}$
SF T2 Before	Average temperature of downstream sapflow temperature sensor (T2) before heat pulse	$^{\circ}\text{C}$
SF T1 During	Average temperature of upstream sapflow temperature sensor (T1) during heat pulse	$^{\circ}\text{C}$
SF T2 During	Average temperature of downstream sapflow temperature sensor (T2) During heat pulse	$^{\circ}\text{C}$
SF T1 After	Average temperature of upstream sapflow temperature sensor (T1) after heat pulse	$^{\circ}\text{C}$
SF T2 After	Average temperature of downstream sapflow temperature sensor (T2) after heat pulse	$^{\circ}\text{C}$
HPV	Estimated HPV from Marshall formula (Eq.1)	m/h (metres per hour)
Bootcount	keeps track of the number of cycles. The value needs to be multiplied by 100 to get the actual boot count	number of cycles

7 Firmware Upgrade

Firmware can be upgraded through two ways: Through a physical connection with a PC, or Over the Air (OTA) using a mobile device or PC. This section provides details on how to accomplish firmware upgrade with either method.

7.1 Configuring PlatformIO and Source Files in VS Code

1. Install the [PlatformIO IDE](#) within [Visual Studio Code](#). For installation instructions, refer to the [PlatformIO IDE for VSCode Documentation](#).
2. Download the WiLo source files from the [WiLo GitHub Repository](#).
3. Unzip the `WiLo-main.zip` file to a location on your PC, and take note of the directory path.
4. Open Visual Studio Code and click on the PlatformIO icon in the left toolbar.
5. In the Quick Access section, click on Open.
6. On the PlatformIO welcome screen, click Open Project. This opens a file navigation window.
7. Navigate to the `WiLo-main/WiLoV2/WiLoV2 Code` directory (where the ZIP file was extracted). Inside the `WiLoV2 Code` folder, ensure there is a `platformio.ini` file. Select the folder and click Open "WiLoV2 Code".
8. A popup will prompt you to trust the authors of the files. Select Yes. PlatformIO will now configure the project, which may take a few minutes.
9. At this point the project files are ready to be flashed to the WiLo.

7.2 Physical Connection Firmware Upgrade

1. After configuration of Section 7.1 is completed, click the PlatformIO icon again on the left toolbar.
2. Connect the ESP32-S2 Mini board to your PC using either a USB-C to USB-A or USB-C to USB-C cable.
3. To enter download mode on the ESP32-S2:
 - Hold down the `0` button (labelled EN or `0`).
 - While holding it, press the RST (Reset) button once.
 - Release the RST button.
 - Finally, release the `0` button.

This sequence is: HOLD EN → PRESS RST → RELEASE RST → RELEASE EN.

4. If done correctly, the device will be ready to flash. On the bottom bar of VS Code, click the plug icon labeled Auto.
5. A dropdown will appear at the top of VS Code. Select the appropriate COM port. It may be labeled similarly to COM6 : `Lolin ESP32-S2 mini Dev Module` (your port number may differ).
6. In the left pane under Project Tasks, click Upload. This will compile the project and flash it to the microcontroller. The Terminal will show flash progress as a percentage.
7. Note: the terminal may end with an error message — this simply indicates that the ESP32-S2 Mini was placed into download mode manually. To run the program:

- Press the RST button again, or
- Power cycle the board by unplugging the USB cable, put the power switch in the off position, switch the WiLo on again and reconnect the USB cable.

7.3 Wireless Connection (OTA) Firmware Upgrade

WiLos can be flashed wirelessly. This requires the `firmware.bin` file to be retrieved beforehand. Note that this is only intended for future WiLo source code changes, as some WiLos might have an older version of the firmware that does not support OTA. If a WiLo was physically flashed with the latest firmware before field deployment, OTA is possible.

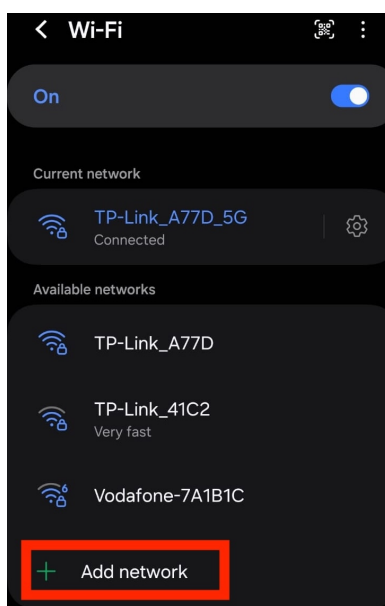
7.3.1 Retrieving `firmware.bin` File

- Ensure that PlatformIO is configured as per Section 7.1.
- Click on the PlatformIO icon on the left tab.
- Under **Project Tasks** click **Build**.
- Once build is completed (it will say **SUCCESS** in the terminal), click **Build** once more. This ensures that the latest version of the firmware is copied.
- In the terminal, you should see a line saying: “Copied firmware.bin to: /Users/.../WiLo Code/firmware/firmware.bin”.

7.3.2 Setting up mobile device/PC for OTA

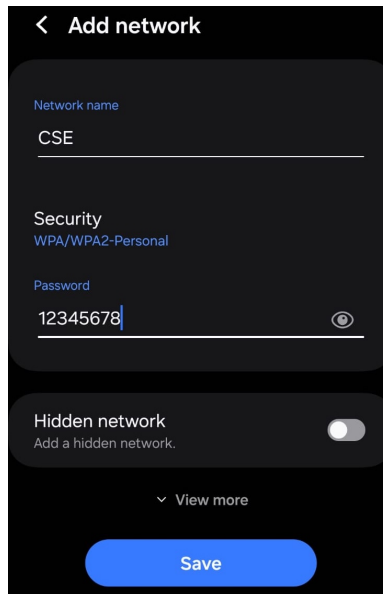
The steps described below are crucial for allowing the WiLo to connect wirelessly. OTA can be done with either a PC or a smartphone.

1. On a PC or smartphone, in the WiFi settings, click "Add network".

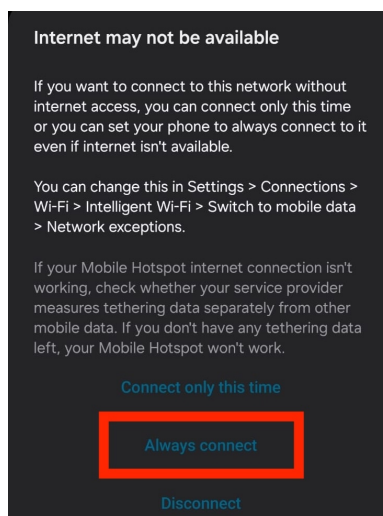


2. Enter the network credentials and click "Save":

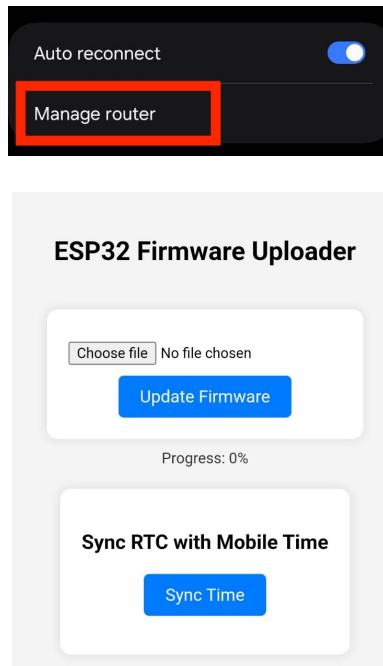
- **SSID:** CSE
- **PASSWORD:** 12345678



3. Copy the `firmware.bin` to the device doing the firmware upgrade.
4. Turn off the WiLo, and turn it on again. This will reset the WiLo, putting it in a WiFi access point state for 15 seconds until a device connects to it. If no device connects within the sixty seconds, it will simply restart, disabling WiFi to conserve power, and continue normal operation.
5. Upon resetting the WiLo, attempt to connect to the WiLo hotspot. You might be prompted with a popup saying that internet may not be available/ Select "Always connect".



6. On the connected device, go to the network settings, and determine the IP-Address of the WiLo WiFi network and paste it in any browser on the mobile device. Alternately, select "Manage router".



7. You will be greeted with the Firmware updaters webserver of the WiLo. You can now browse the mobile device storage to upload firmware (`firmware.bin`), and sync the RTC.
8. Click on “Choose file”, select the `firmware.bin` file, and click on “Update Firmware”. The progress bar will show the progress completion percentage of the Update.
9. When the update is completed, the WiLo will restart itself and a new WiFi window will be opened. You can simply disconnect from the WiLo, or turn off the WiFi of the connected device. The WiLo will restart one final time, and the new firmware will be in place for normal operation to continue.

8 Debugging

This section will describe some key hardware debugging tips.

8.1 Hardware Debugging

8.1.1 Battery

If the battery level is at around 10%, the WiLo immediately goes into deep sleep for some time (15 min). This reduces power consumption, in the attempt of allowing the battery to recharge via solar before sensor measurements are taken, or a payload is transmitted. However, the batteries may struggle to recharge sufficiently long term when at this critical level, especially during low-sunlight conditions. Therefore, it is the technician’s responsibility to swap out the battery for a fully charged battery in this instance.

To measure the battery voltage, a multimeter is required. Place the positive and negative probes of the multimeter on the positive and negative terminals of the battery. Correspond the voltage read by the multimeter to a SoC% as shown in Table 8.

Table 8: Battery Voltage vs State of Charge (10% increments)

SoC (%)	Battery Voltage (V)
100	4.200
90	3.936
80	3.847
70	3.803
60	3.769
50	3.737
40	3.698
30	3.685
20	3.635
10	3.350
0	3.000

8.1.2 SD Card Errors

In some cases the SD card will not be recognised by the WiLo. This will result in a loop where the debug LED will flash. This likely is due to the SD card not being in the correct format. Refer to Section 6 for information on SD card formatting.

8.2 Sensor Errors

The following table will describe the error codes used to determine whether sensors might be faulty/not properly connected. The error codes are stored in the `Measurements.csv` file in the expected `measurmnt` column of the faulty sensor.

Table 9: Sensor Error Codes and Troubleshooting

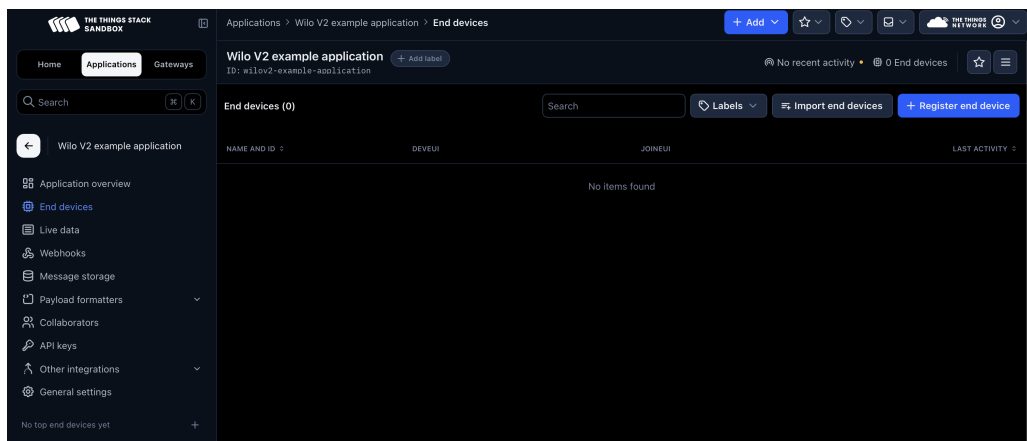
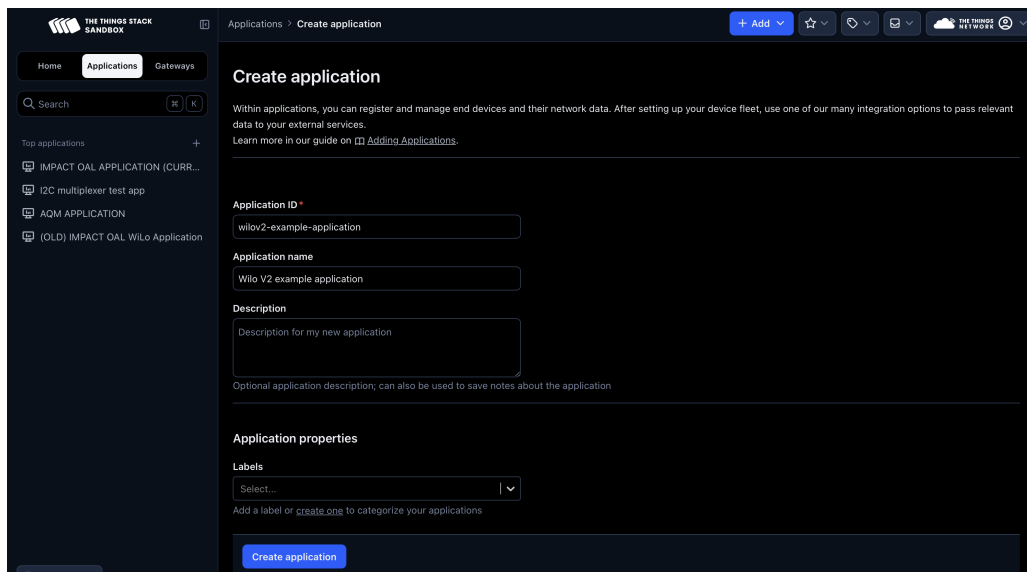
Sensor	Error Code	What to do
DHT22	1111.11 (Temp/Hum out of bounds: $T < -40^{\circ}C$ or $T > 80^{\circ}C$, $H < 0\%$ or $H > 100\%$) 9999.99 (Unresponsive/Not connected: $T = \text{NaN}$ or $H = \text{NaN}$)	Replace sensor Check connections/Replace
SM	Voltage reading stays at 2600mV	Replace sensor
DS18B20 Soil Temp	1111.11 (Temp out of bounds: $T < -40^{\circ}C$ or $T > 80^{\circ}C$) 9999.99 (Unresponsive/Not connected: $T = \text{NaN}$)	Replace sensor Replace
Sapflow Sensor	1111.11 (Not connected/Faulty)	Check connections/Replace
Dendrometer	No error codes in current firmware	N/A
SDI-12	No error codes in current firmware	N/A

9 Adding WiLos to The Things Network

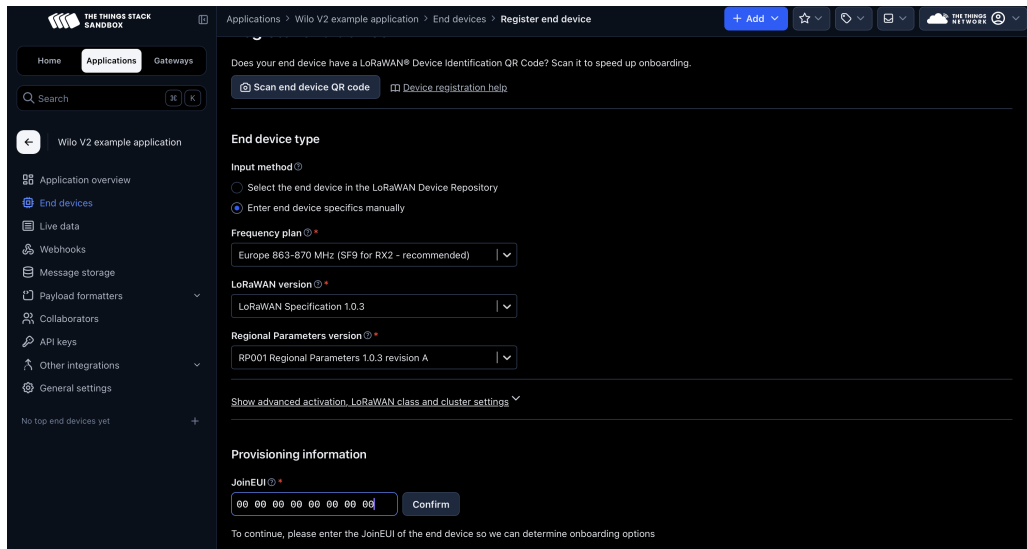
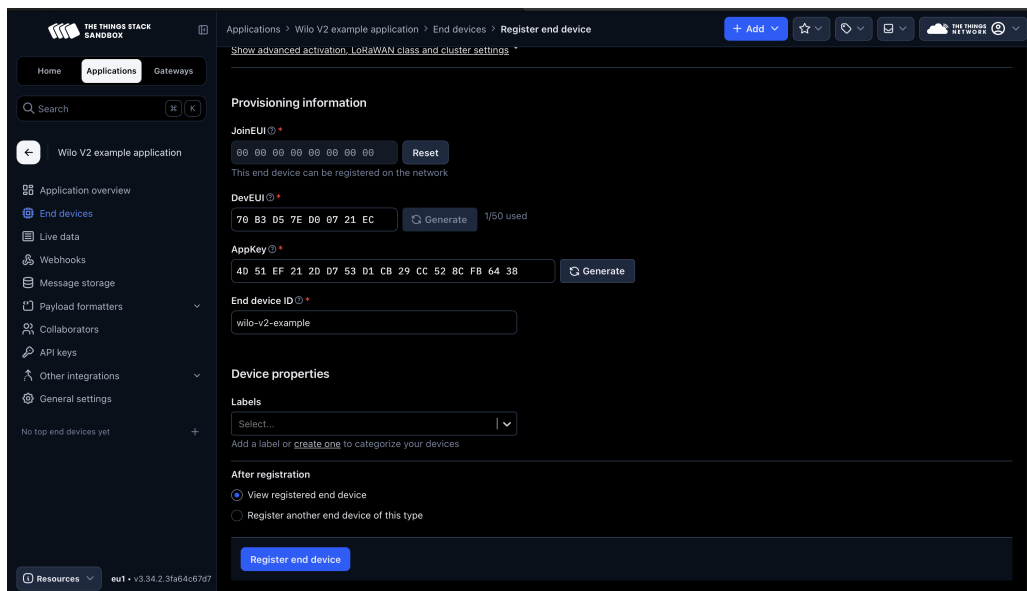
The following section will demonstrate how to navigate TTN in order to add an application, and add WiLo devices to the application.

9.1 Adding a single WiLo

1. Log in at <https://eu1.cloud.thethings.network/console/applications>. If you do not have an account, create one.
2. Click on “Add Application” if you do not have an application that you would like to use yet. Give it a unique ID and a name. If you already have an application, you can skip this step.



- Click on the application you would like to add devices to, and navigate to “End Services” → “Register end device”.
- Under “Input method”, select “Enter device specifics manually”, and select the specifics according to the image below. Populate the JoinEUI with 0000000000000000 and click confirm.
- Generate a DevEUI and AppKey, and give the device a unique name. If it is a new application, make the ID wilo-001. This step is important for consistency between the ID stored on the SD card of WiLo you will be using. Select on “View registered end device” and finally click on “Register end device”.

6. You have successfully added a WiLo entry to the end devices of your application. You can now populate more WiLo loggers by following the same process. Keep in mind, for each WiLo added, the parameters must be stored on the SD card.
7. On a PC, create a “parameters.txt” file, and copy the parameters from the device information page on TTN in the following format:


```

DEVID=1

APPEUI=00000000000000000000

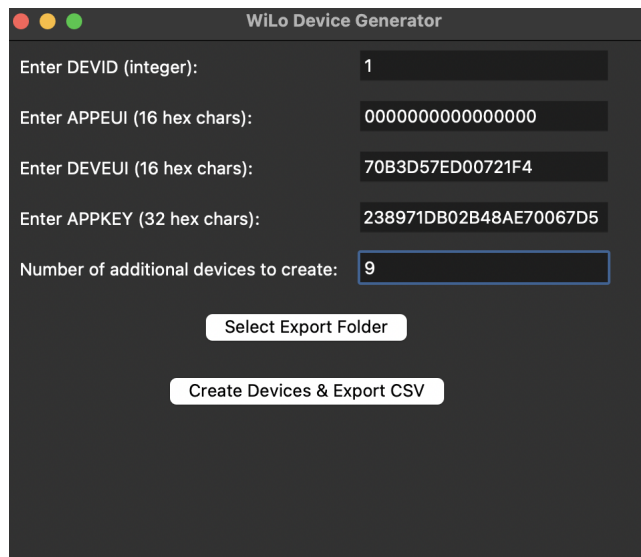
DEVEUI=70B3D57ED00721EC

APPKEY=4D51EF212DD753D1CB29CC528CFB6438
      
```
8. Save the parameters.txt file on the SD card of the WiLo.

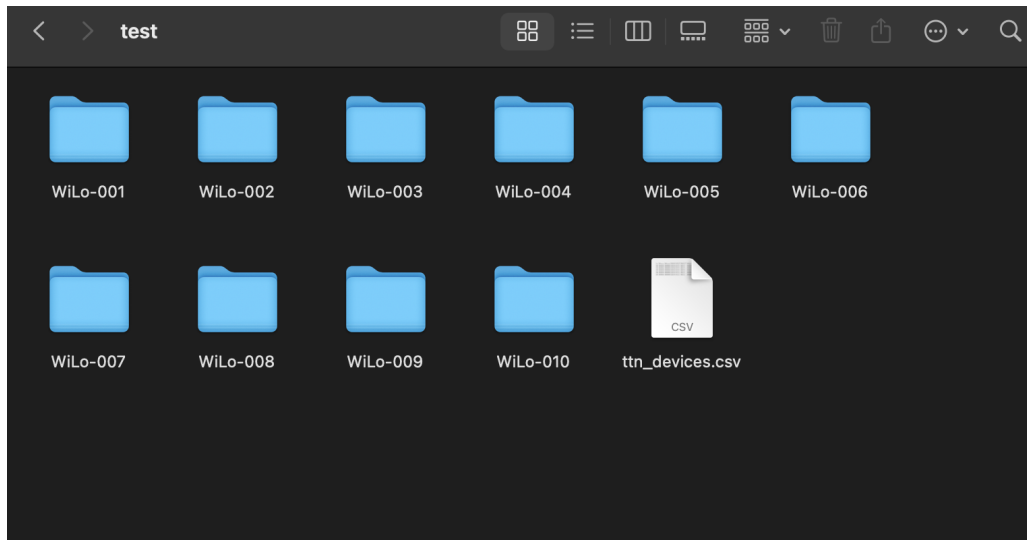
9.2 Bulk Adding Wilos

Manually copying WiLo LoRa parameters is a tedious process; therefore it is recommended that the provided tool in the GitHub repository is used to populate multiple WiLos. A WiLo that is already registered on an application on TTN is required for this process. Note that TTN works on a basis of adding end devices sequentially. Therefore if someone else manually registers a device, the next available number will be generated for them. It is therefore recommended that when multiple WiLos are to be added, it is done immediately after creating the latest WiLo on the application. To bulk add WiLos, follow the following steps:

1. Navigate to WiLo/LoRa key generator/Lora key generator apps. Open the app folder relating to your OS (Windows or MACOS). Run the LKG app.
2. Enter the parameters:
 - **ID** – The unique integer ID relating to the ID you gave the WiLo on TTN. If it is the first, simply enter “1”.
 - **APPEUI, DEVEUI, APPKEY** – Copy these directly from TTN.
 - Choose a number of additional devices to create. In the example, 9 was used (for a total of 10 WiLos).



3. Select a target folder you would like the parameter files to be stored in. In this case it is a folder named “Test” on the desktop.
4. Export the device parameter backups and the bulk add CSV. Once completed, the selected folder should contain the folders of the LoRa parameters corresponding to the number of WiLos, and the bulk add CSV.



5. On TTN, navigate to Applications → Your application → End devices and click on “Import end devices”.
6. Select the .csv file in the location it was stored in. Set the parameters exactly as shown in this example:

A screenshot of the 'Import end devices' form in the TTN console. The form has a dark theme. At the top, it says 'Format information' and 'File containing end devices in The Things Stack CSV format.' Below this is a 'File' section with a 'Change file...' button and the selected file 'ttn_devices.csv'. There is a collapsed section for 'Advanced end device claiming settings'. The 'Fallback values' section contains a warning message and three radio button options: 'Do not set any fallback values', 'Load end device profile from the LoRaWAN Device Repository', and 'Enter LoRaWAN versions and frequency plan manually' (which is selected). Below this are three dropdown menus: 'Frequency plan' set to 'Europe 863-870 MHz (SF9 for RX2 - recommended)', 'LoRaWAN version' set to 'LoRaWAN Specification 1.0.3', and 'Regional Parameters version' set to 'RP001 Regional Parameters 1.0.3 revision A'. At the bottom is a large blue button labeled 'Import end devices'.

7. finally, click on “Import end devices”. A program on TTN will run and automatically add the end devices. There will be a warning stating that one device import was unsuccessful. That is because you have already registered the device.



Notes: In the event of an unsuccessful import of some or all devices, it is likely that the DEVEUI parameters have already been registered by another TTN user. In this case, manually add those devices.

Appendices

Appendix A: TTN Payload Formatter (WiLo V2)

WiLo V2 devices send measurements in compact 3-byte encoded values via LoRaWAN. Each physical quantity is encoded as:

$$\text{Value} = [\text{Byte } 1] \times 100 + [\text{Byte } 2] + \frac{[\text{Byte } 3]}{100}$$

This encoding allows accurate float representation in a compact binary format.

Decoder Logic

The JavaScript payload formatter for The Things Network (TTN) decodes the payload as follows:

1. Parses fixed values from onboard sensors and sapflow sensors.
2. Uses 3 metadata bytes at the end of the payload:
 - **WiLoConfig**: Encodes presence of MUX and SDI-12 sensor type.
 - **SensorsTypeOnPorts**: Specifies type of sensor on each of the 8 MUX ports (bitfield).
 - **PortsOfMuxConnected**: Indicates which MUX ports are populated (bitfield).
3. Conditionally decodes:
 - SDI-12 sensors:
 - **CS655**: 2 × soil moisture, 1 × temperature.
 - **Sentek 60 cm**: 6 × moisture, 6 × temperature.
 - **Sentek 90 cm**: 9 × moisture, 9 × temperature.
 - MUX: Up to 8 ports (P0–P7), each providing either:
 - A Dendrometer reading (P#_Dendro); or
 - Sapflow data: T1B, T2B, T1D, T2D, T1A, T2A per port. (T1 = temperature at sensor 1, T2 = Temperature at sensor 2, B= Before heatpulse, D = During heatpulse, A = after heatpulse).

Fixed Fields (13 values = 39 bytes)

MUX Field Decoding

If the MUX is connected (bit 0 of WiLoConfig set), the decoder checks each of the 8 ports (P0–P7):

- If bit n of PortsOfMuxConnected is set, port P_n is active.
- If bit n of SensorsTypeOnPorts is also set:
 - Decode a dendrometer reading (P#_Dendro).
- Else:
 - Decode 6 sapflow temperatures per port: T1B, T2B, T1D, T2D, T1A, T2A.

Field	Description
Dendro	Onboard dendrometer reading (μm)
AirTemp	Air temperature (DHT22, $^{\circ}\text{C}$)
AirHumidity	Air relative humidity (DHT22, %)
SoilTemp	Soil temperature (18B20S, $^{\circ}\text{C}$)
SoilMoistureOrPyranometer	Soil moisture (Capacitive) or solar irradiance (Pyranometer)
BatteryLevel	Battery percentage remaining (%)
T1Before, T2Before	Sapflow pre-heat pulse temperatures ($^{\circ}\text{C}$)
T1During, T2During	Sapflow during-heat pulse temperatures ($^{\circ}\text{C}$)
T1After, T2After	Sapflow post-heat pulse temperatures ($^{\circ}\text{C}$)
BootCount	Reboot counter (converted back to integer)

Table 10: Standard WiLo V2 payload fields

SDI-12 Sensor Decoding

The SDI-12 sensor type is encoded in bits 1 and 2 of `WiLoConfig`. One of the following options is selected:

- `0b10` (Sentek 60 cm): `SDI12_Moisture_1-6`, `SDI12_Temp_1-6`
- `0b100` (Sentek 90 cm): `SDI12_Moisture_1-9`, `SDI12_Temp_1-9`
- `0b110` (CS655): `SDI12_Moisture_1-2`, `SDI12_Temp_1`
- Else: No SDI-12 sensor connected

Each SDI-12 measurement is 3 bytes and decoded similarly to fixed fields.

Metadata Byte Summary

Byte	Name	Description
<code>b[length - 3]</code>	<code>WiLoConfig</code>	Bitfield for SDI-12 type + MUX present
<code>b[length - 2]</code>	<code>SensorsTypeOnPorts</code>	8-bit map: 1 = dendrometer, 0 = sapflow
<code>b[length - 1]</code>	<code>PortsOfMuxConnected</code>	8-bit map: 1 = port active

Table 11: Payload metadata byte mapping

NOTE: Only a single SDI-12 sensor can be connected to a WiLo at any given time. There won't be any cases other than the ones mentioned above. Not all WiLos will have a SDI-12 sensor connected, and will fall under the default case.

Payload formatter code

The payload formatter can be found in the GitHub repository, in the `PayloadFormatterBackup` folder. The `README.MD` file provides instructions on how to integrate the payload formatter into TTN.

```
function decodeUplink(input) {
  const b = input.bytes;
  const port = input.fPort;
  const payloadLength = b.length;
  const decoded = {};
  const warnings = [];
  let i = 0;
```

```

const WiLoConfig = b[payloadLength - 3];
const SensorsTypeOnPorts = b[payloadLength - 2];
const PortsOfMuxConnected = b[payloadLength - 1];

function isBitSet(value, bitPosition) {
  return (value & (1 << bitPosition)) !== 0;
}

function isSDI12Connected(config) {
  const mask = 0b00000110;
  const DD_60CM = 0b00000010;
  const DD_90CM = 0b00000100;
  const CS655 = 0b00000110;
  const maskedbyte = config & mask;

  switch (maskedbyte) {
    case DD_60CM: warnings.push("Sentek 60cm D&D connected"); return 1;
    case DD_90CM: warnings.push("Sentek 90cm D&D connected"); return 2;
    case CS655:   warnings.push("CS655 connected"); return 3;
    default:      warnings.push("No SDI-12 connected"); return 0;
  }
}

const SDI12State = isSDI12Connected(WiLoConfig);
const muxConnected = isBitSet(WiLoConfig, 0);
if (muxConnected) warnings.push("MUX is connected");

// === Fixed fields ===
decoded.Dendro = b[i] * 100 + b[i + 1] + b[i + 2] / 100; i += 3;
decoded.AirTemp = b[i] * 100 + b[i + 1] + b[i + 2] / 100; i += 3;
decoded.AirHumidity = b[i] * 100 + b[i + 1] + b[i + 2] / 100; i += 3;
decoded.SoilTemp = b[i] * 100 + b[i + 1] + b[i + 2] / 100; i += 3;
decoded.SoilMoistureOrPyranometer = b[i] * 100 + b[i + 1] + b[i + 2] / 100; i += 3;
decoded.BatteryLevel = b[i] * 100 + b[i + 1] + b[i + 2] / 100; i += 3;
decoded.T1Before = b[i] * 100 + b[i + 1] + b[i + 2] / 100; i += 3;
decoded.T2Before = b[i] * 100 + b[i + 1] + b[i + 2] / 100; i += 3;
decoded.T1During = b[i] * 100 + b[i + 1] + b[i + 2] / 100; i += 3;
decoded.T2During = b[i] * 100 + b[i + 1] + b[i + 2] / 100; i += 3;
decoded.T1After = b[i] * 100 + b[i + 1] + b[i + 2] / 100; i += 3;
decoded.T2After = b[i] * 100 + b[i + 1] + b[i + 2] / 100; i += 3;
decoded.BootCount = b[i] * 100 + b[i + 1] + b[i + 2] / 100; i += 3;
decoded.BootCount = decoded.BootCount*100; // convert back to an integer ranging between 1 and 999999

// === MUX decoding ===
if (muxConnected) {
  for (let port = 0; port < 8; port++) {
    const portLabel = `P${port}`;
    if (isBitSet(PortsOfMuxConnected, port)) {
      if (isBitSet(SensorsTypeOnPorts, port)) {
        if (i + 2 < payloadLength - 3) {
          decoded[`${portLabel}_Dendro`] = b[i] * 100 + b[i + 1] + b[i + 2] / 100;
          i += 3;
        }
      } else {
        const labels = ["T1B", "T2B", "T1D", "T2D", "T1A", "T2A"];
        for (const label of labels) {
          if (i + 2 < payloadLength - 3) {
            decoded[`${portLabel}_${label}`] = b[i] * 100 + b[i + 1] + b[i + 2] / 100;
            i += 3;
          }
        }
      }
    }
  }
}

// === SDI-12 decoding ===
switch (SDI12State) {
  case 1: // 60cm Sentek
    for (let ch = 1; ch <= 6; ch++) {
      if (i + 2 < payloadLength - 3) {
        decoded[`SDI12_Moisture_${ch}`] = b[i] * 100 + b[i + 1] + b[i + 2] / 100;

```

```

        i += 3;
    }
}
for (let ch = 1; ch <= 6; ch++) {
    if (i + 2 < payloadLength - 3) {
        decoded['SDI12_Temp_${ch}'] = b[i] * 100 + b[i + 1] + b[i + 2] / 100;
        i += 3;
    }
}
break;
case 2: // 90cm Sentek
    for (let ch = 1; ch <= 9; ch++) {
        if (i + 2 < payloadLength - 3) {
            decoded['SDI12_Moisture_${ch}'] = b[i] * 100 + b[i + 1] + b[i + 2] / 100;
            i += 3;
        }
    }
    for (let ch = 1; ch <= 9; ch++) {
        if (i + 2 < payloadLength - 3) {
            decoded['SDI12_Temp_${ch}'] = b[i] * 100 + b[i + 1] + b[i + 2] / 100;
            i += 3;
        }
    }
    break;
case 3: // CS655
    for (let ch = 1; ch <= 2; ch++) {
        if (i + 2 < payloadLength - 3) {
            decoded['SDI12_Moisture_${ch}'] = b[i] * 100 + b[i + 1] + b[i + 2] / 100;
            i += 3;
        }
    }
    if (i + 2 < payloadLength - 3) {
        decoded['SDI12_Temp_1'] = b[i] * 100 + b[i + 1] + b[i + 2] / 100;
        i += 3;
    }
    break;
}

return {
    data: decoded,
    warnings: warnings,
    errors: []
};
}

```

Appendix B: PCB Layout and Schematic

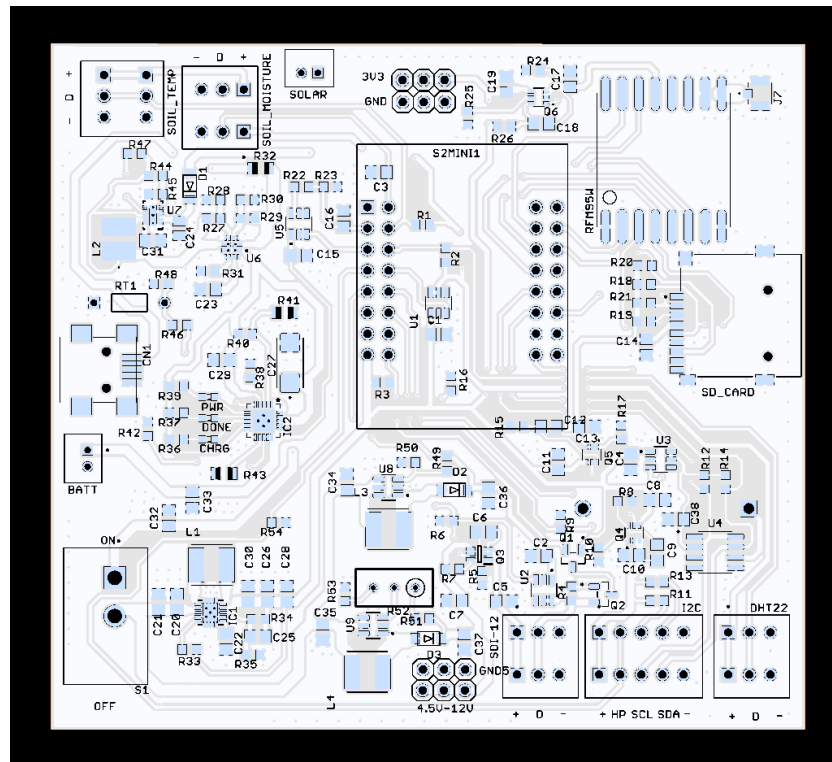


Figure 5: Top Layer of the WiLo V2 PCB

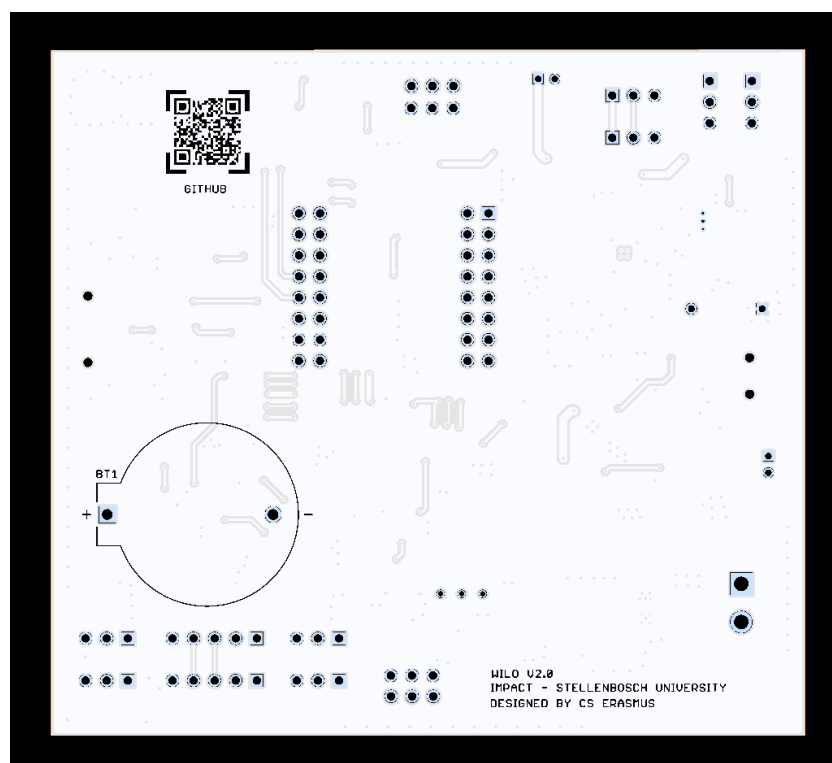


Figure 6: Bottom layer of the WiLo V2 PCB

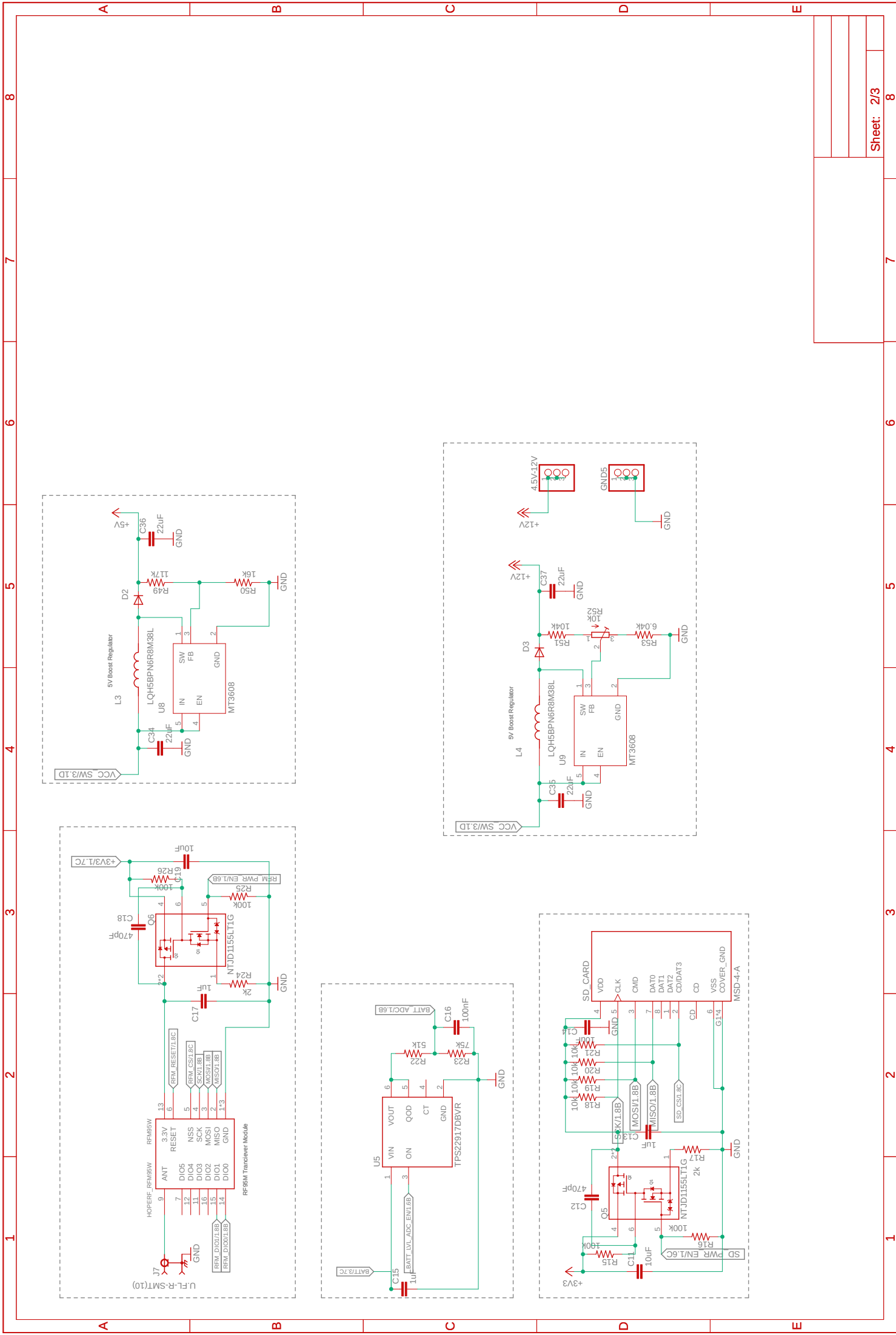


Table 12: ESP32-S2 Mini pin mapping for WiLo V2 (Wireless Logger). Pin names correspond to ‘#define’ labels used in firmware.

Pin Name	Description	ESP32-S2 Mini Pin
Environmental Sensors		
DHT22_DATA_PIN	Data pin for DHT22 sensor	GPIO6
SM_ADC_PIN	Analog signal from capacitive soil moisture sensor	GPIO1
DHT22_SM_ST_EN_PIN	Enables power to DHT22, soil moisture, and ST sensors	GPIO8
SOIL_TEMPERATURE_DATA_PIN	Data pin for DS18B20 soil temperature sensor	GPIO2
Battery Sensing		
BATTERY_LVL_ADC_EN_PIN	Enables voltage divider for battery sensing	GPIO5
BATTERY_ADC_PIN	Reads battery voltage via ADC	GPIO3
Sapflow & Dendrometer (I2C Port)		
HEAT_PULSE_EN_PIN	Enables sapflow heat pulse	GPIO7
I2C_PORT_EN_PIN	Powers I2C port (DigiDendro/sapflow/MUX)	GPIO13
I2C_SDA	I2C data line	GPIO17
I2C_SCL	I2C clock line	GPIO21
SDI-12 Sensors		
SDI12_DATA_PIN	Data pin for SDI-12 sensors	GPIO14
SDI12_EN_PIN	Powers SDI-12 sensors	GPIO9
LoRa RFM95W (SPI1)		
LORA_EN_PIN	Enables LoRa module	GPIO4
LORA_SCK_PIN	SPI clock line for LoRa	GPIO36
LORA_MISO_PIN	SPI MISO for LoRa	GPIO37
LORA_MOSI_PIN	SPI MOSI for LoRa	GPIO35
LORA_CS_PIN	Chip select for LoRa module	GPIO34
LORA_RST_PIN	Reset pin for LoRa module	GPIO18
LORA_DIO0_PIN	Interrupt pin 0 for LoRa	GPIO33
LORA_DIO1_PIN	Interrupt pin 1 for LoRa	GPIO38
LORA_SPI	SPI bus number for LoRa	SPI1
SD Card Interface (SPI2)		
SD_SCK_PIN	SPI clock line for SD card	GPIO36
SD_MISO_PIN	SPI MISO for SD card	GPIO37
SD_MOSI_PIN	SPI MOSI for SD card	GPIO35
SD_CS_PIN	Chip select for SD card	GPIO16
SD_ENABLE_PIN	Enables power to SD card	GPIO10
SD_SPI	SPI bus number for SD card	SPI2
Miscellaneous		
RTC_EN_PIN	Powers RTC module	GPIO15
LED_PIN	Onboard debug/status LED	GPIO15